

iterativeDecomp.pdf



abcd__1234



Paralelismo



3º Grado en Ingeniería Informática



Facultad de Informática de Barcelona (Fib)
Universidad Politécnica de Catalunya



[Accede al documento original](#)



Una cuenta que no te pide **nada**.

Ni siquiera que apruebes. De momento.
(Estudia y no nos des ideas)

Cuenta NoCuenta

[Saber más](#)

1/6
Este número es indicativo del riesgo del producto, siendo 1/6 indicativo de menor riesgo y 6/6 de mayor riesgo.

ING BANK/Nº se encuentra adherido al Sistema de Garantía de Depósitos, regulado con una garantía de hasta 100.000 euros por depositante. Consulta más información en [ing.es](#)



Ya has abierto los apuntes,
te mereces ese descanso.

También te mereces que no te cobren
por tener una cuenta. **Cositas.**

Ven a la **Cuenta NoCuenta**

Saber más



Pregunta 1

Completa

Puntuació 1,00 sobre 1,00

[Marca la pregunta](#)

What kind of OpenMP implementation will you use to parallelize a countable loop like the one shown below? (assuming that you cannot modify the sequential version of the code)

```
for (int i = 0; i < n; i++)  
    C[i] = A[i] + B[i];
```

Trieu-ne una o més:

- ☐ We can ONLY use the implicit tasks generated in «#pragma omp parallel», assigning to each of them a subset of the iterations with the «omp_get_thread_num» and «omp_get_num_threads» intrinsic functions.
- ☒ We can use explicit tasks by inserting «#pragma omp task» in the loop body, one task per iteration of the loop; however it may suffer of too small granularity. The code should be executed within the scope of «#pragma omp parallel» and «#pragma omp single».
- ☒ We can use explicit tasks using «#pragma omp taskloop» with an appropriate task granularity (default or set by using the grainsize or num_tasks clauses). The code should be executed within the scope of «#pragma omp parallel» and «#pragma omp single»

WUOLAH

Pregunta 2

Completa

Puntuació 1,00 sobre 1,00

 [Marca la pregunta](#)

Which OpenMP constructs will you use to implement an iterative task decomposition for the following code?

```
void traverse_list(List l) {  
    Element e;  
    for (e = l->first; e; e = e->next)  
        process(e);  
}  
  
void main() {  
    traverse_list(ll);  
}
```

Trieu-ne una o més:

- ☐ Just adding «#pragma omp parallel» in the loop body, enabling in this way the execution of multiple invocations of function process as implicit tasks.
- ☐ We need to add «#pragma omp parallel» and «#pragma omp single» before calling function traverse_list from main and then «#pragma omp taskloop» just in front of the for loop.
- ☒ We need to add «#pragma omp parallel» and «#pragma omp single» before calling function traverse_list from main and then «#pragma omp task» just in front of the process function invocation.
- ☐ Independently of the option chosen above, we also need to introduce some kind of synchronization to protect

Pregunta 3

Completa

Puntuació 1,00 sobre 1,00

 Marca la pregunta

Assume the following four versions of a function (in the scope of a parallel region with single) counting the number of positive values (including zero) in a vector (with a sufficiently large number of elements n), all versions using a linear iterative task decomposition:

Code 1)

```
int it_hist_pos (int *vector, int n) {
    int pos = 0;
    #pragma omp taskloop shared(pos)
    for (int i = 0; i < n; i++)
        if (vector[i] >= 0) {
            #pragma omp critical
            pos++;
        }
    return pos;
}
```

Code 2)

```
int it_hist_pos (int *vector, int n) {
    int pos = 0;
    #pragma omp taskloop shared(pos)
    for (int i = 0; i < n; i++)
        if (vector[i] >= 0) {
            #pragma omp critical(positives)
            pos++;
        }
    return pos;
}
```




```
    if (vector[i] >= 0)
        pos++;
    return pos;
}
```

Which of the following statements establishes the appropriate order of the four codes above in terms of performance, from higher to lower:

Trieu-ne una:

- ☐ Code 4 > Code 3 > Code 2 >= Code 1, taking into account both the number and kind of synchronisations performed in each case. Between Code 2 and 1 it depends on the existence of other critical regions in the program.
- ☐ Code 3 > Code 4 > Code 2 > Code 1 since atomic incurs a negligible overhead thanks to hardware support.
- ☐ Code 2 > Code 3 > Code 4 > Code 1 since named critical regions allow a better isolation of data races, even better than atomic that only protect the read-update-write of a shared variable.
- ☐ Code 1 > Code 2 > Code 3 > Code 4 since unnamed critical has less overhead than named critical. Both kind of critical regions have less restrictions than atomic, which is also used in the implementation of the reduction clause.

Pregunta 4

Completa

Puntuació 1,00 sobre 1,00

 [Marca la pregunta](#)

Assume the following code that inserts the elements of vector ToInsert in a hash table (defined as a vector of linked lists) named HashTable making use of locks to protect the access to it:

```
#define SIZE_TABLE ...
#define MAX_ELEM ...
int ToInsert[MAX_ELEM];

#define SIZE_TABLE 32384
typedef struct {
    int data;
    element * next;
} element;

element * HashTable[SIZE_TABLE];

omp_lock_t lock_vector[<<size>>];

int main() {
    ...
    #pragma omp parallel
    #pragma omp single
    for (long i = 0; i < MAX_ELEM; i++) {
        #pragma omp task <<task-clause>>
        {
            int index = hash_function(ToInsert[i], SIZE_TABLE); /* not intensive */
```

WUOLAH



8-11 JULY 2026

8th WEDNESDAY

FOO FIGHTERS · MOBY · THE WAR ON DRUGS · WOLF ALICE
THE LAST DINNER PARTY · THE WARNING · JEHNNY BETH · THE WAR AND TREATY
PALAYE ROYALE · DOGSTAR · HOT MILK · VILLANELLE · HOTWAX
HOONINE · MADMADMAD · BIGGER SPLASH

9th THURSDAY

FLORENCE + THE MACHINE · JENNIE · LORDE · TEDDY SWIMS
ZARA LARSSON · RENÉE RAPP · CHARLIE PUTH · THE BLAZE dj set · CMAT
SADIE JEAN · ZIMMER90 · CHLOE SLATER · FROST CHILDREN · SON MIEUX · FLORENTENES · LA PALOMA
BOYS NOIZE · PALMS TRAX · YUNG PRADO · AMRITA THE LOOP

10th FRIDAY

TWENTY ONE PILOTS · KINGS OF LEON · PIXIES · HALSEY
A PERFECT CIRCLE · INTERPOL · SIGRID · HOLLY HUMBERSTONE · MIDNIGHT GENERATION
CLIFFORDS · USTED SEÑALEMELO · KAREN DIÓ · RIO KOSTA · MY FIRST TIME
POLO & PAN dj set · BUNT. · SWIMMING PAUL · WEVAL · MAESIC THE LOOP

11th SATURDAY

NICK CAVE & THE BAD SEEDS · PULP · THE BLACK CROWES · DAVID BYRNE
KASABIAN · JALEN NGONDA · THE VACCINES · MATT BERNINGER
THE REYTONS · PERSIA HOLDER · OVERPASS · TTSSFU · SUPERMODEL*
RICHIE HAWTIN · NINA KRAVIZ · AEREA · LUXI VILLAR THE LOOP

TICKETS ON SALE NOW!

www.madcooltickets.com



The implementation of function `insert_elem` does not allow multiple instances of it running in parallel for the same value of `index`. Which of the following statements are true?

Tríeú-ne una o m s:

- ☒ «size» could be equal to 1. In this case «statement-1» and «statement-2» should simply set and unset, respectively, the lock variable `lock_vector[0]`. However, this strategy is not exploiting the whole parallelism in the parallel region.
- ☐ «size» equal to `MAX_ELEM`. In this case «statement-1» and «statement-2» should simply set and unset, respectively, the lock variable `lock_vector[i]`; in this way each iteration of the loop can do the insertion of `ToInsert[i]` without any chance for data races.
- ☒ «size» equal to `SIZE_TABLE`. In this case «statement-1» and «statement-2» should simply set and unset, respectively, the lock variable `lock_vector[index]`; in this way multiple iterations of the loop will compete to do insertions in the same entry of the hash table.
- ☒ «task-clause» can be left empty. In this code, by default the compiler will capture the value for all the variables that are needed to execute each iteration of the loop as a task.
- ☐ «task-clause» should be `shared(i)`. By default the compiler will make variable `i` firstprivate; this is not the appropriate behaviour since each task needs to access to the shared variable `ToInsert[i]`.

Pregunta 5

Completa

Puntuació 1,00 sobre 1,00

 [Marca la pregunta](#)

Expert

2 DÍAS
CONSECUTIVOS**86€**

Long expert

5 DÍAS
DE LIBRE USO**185€**Más info.:
elsubidon.es

Assume the following code that inserts the elements of vector ToInsert in a hash table (defined as a vector of lists) named HashTable making use of task dependences the access to it:

```
#define SIZE_TABLE ...
#define MAX_ELEM ...
int ToInsert[MAX_ELEM];

#define SIZE_TABLE 32384
typedef struct {
    int data;
    element * next;
} element;
element * HashTable[SIZE_TABLE];

int main() {
    ...
    #pragma omp parallel
    #pragma omp single
    for (long i = 0; i < MAX_ELEM; i++) {
        int index = hash_function(ToInsert[i], SIZE_TABLE); /* not intensive */
```

WUOLAH

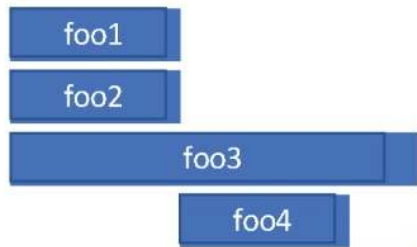
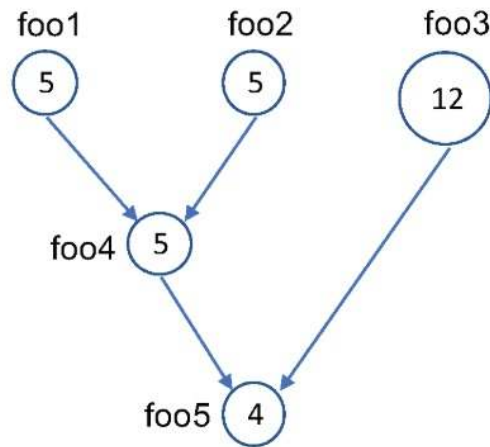
```
}
```

Which of the following statements is true?

Tríeú-ne una:

- ☐ The task definition in this code is not correct. A task should include the two statements in the loop body, i.e. the invocations of `hash_function` and `insert_element`. With that changed, «depend-clause» should be «depend(inout:index)». «data-clause» could be left empty since the defaults appropriately handle all variables used in the task.
- ☐ The task definition in this code is correct. In order to exploit maximum parallelism, «depend-clause» should be «depend(inout:index)» and «data-clause» could be left empty since the defaults appropriately handle all variables used in the task.
- ☒ The task definition in this code is correct. In order to exploit maximum parallelism, «depend-clause» should be «depend(inout:HashTable[index])» and «data-clause» could be left empty since the defaults appropriately handle all variables used in the task.

Consider the TDG (expressing dependencies between tasks) and execution timeline for the tasks on the left of the following figure:



Code 1

```
#pragma omp task
foo1()
#pragma omp task
foo2()
#pragma omp taskwait
#pragma omp task
foo3()
#pragma omp task
foo4()
#pragma omp taskwait
#pragma omp task
foo5()
```

Code 2

```
#pragma omp task
foo1()
#pragma omp task
foo2()
#pragma omp task
foo3()
#pragma omp taskwait
#pragma omp task
foo4()
#pragma omp taskwait
#pragma omp task
foo5()
```

Code 3

```
#pragma omp task
{
  #pragma omp task
  foo1()
  #pragma omp task
  foo2()
  #pragma omp taskwait
  #pragma omp task
```

Code 4

```
#pragma omp task
foo3()
#pragma omp taskgroup
{
  #pragma omp task
  foo1()
  #pragma omp task
  foo2()
```


2020
26
8-11 JULY

MAD
COOL
FESTIVAL 10

CONSIGUE
TU ENTRADA
AQUÍ →



```
Code 1:
#pragma omp taskwait
}
#pragma omp task
foo3()
#pragma omp taskwait
#pragma omp task
foo5()

Code 2:
#pragma omp taskwait
foo4()
#pragma omp taskwait
#pragma omp task
foo5()

Code 3:
#pragma omp taskwait
foo4()
#pragma omp taskwait
#pragma omp taskwait
foo5()

Code 4:
#pragma omp taskwait
foo4()
#pragma omp taskwait
#pragma omp taskwait
foo5()
```

Which of the four code above would achieve the parallelism that is available in the TDG?

Tríeune una o más:

- ☐ None of the codes above achieves the potential parallelism that is available in the TDG.
- ☐ Code 1
- ☒ Code 2
- ☐ Code 3
- ☒ Code 4

WUOLAH